

Mushroom ML

Classificazione Funghi Velenosi / Commestibili

Dataset UCI Mushroom · Logistic Regression · XGBoost · Flask API
Andrea Morlotti Riccardo Scampini

Introduzione

Pipeline completo

Load → EDA → Split → Clean → Encode → Train → Evaluate

Anti data leakage

Fit sempre su X_{train} , transform su train e test separatamente

Due modelli

Logistic Regression (baseline lineare) e XGBoost (ensemble boosting)

Flask API REST

Ogni step è esposto come endpoint chiamabile in sequenza

UCI Mushroom Dataset

8.124

campioni totali

21

feature totali

2

classi target

30,5%

valori mancanti stalk-root

Colonna	Tipo	Note
poisonous (target)	categoriale	e = edibile, p = velenoso
stalk-root	categoriale	30% mancanti → riempiti 'unknown'
veil-type	categoriale	rimossa (un solo valore)
odor	categoriale	feature molto predittiva

Dizionario delle Feature (1 / 2)

cap-shape

Forma del cappello

b=campanula, c=conico, x=convesso, f=piatto, k=nodoso, s=incavato

cap-color

Colore del cappello

n=marrone, b=beige, c=cannella, g=grigio, r=verde, p=rosa, u=viola, e=rosso, w=bianco, y=giallo

odor

Odore del fungo

a=mandorla, l=anice, c=catrame, y=pesce, f=putrido, m=muffa, n=nessuno, p=pungente, s=speziato

gill-spacing

Spaziatura delle lamelle

c=vicine, w=affollate, d=distanti

gill-color

Colore delle lamelle

k=nero, n=marrone, b=beige, h=cioccolato, g=grigio, r=verde, o=arancio, p=rosa, u=viola, e=rosso, w=bianco, y=giallo

cap-surface

Superficie del cappello

f=fibroso, g=scanalato, y=squamoso, s=liscio

bruises

Presenza di ammaccature

t=sì, f=no

gill-attachment

Attacco delle lamelle al gambo

a=attaccate, d=discendenti, f=libere, n=intaccate

gill-size

Dimensione delle lamelle

b=larghe, n=strette

stalk-shape

Forma del gambo

e=allargato, t=affusolato

Dizionario delle Feature (2 / 2)

stalk-root

Radice del gambo (30% mancanti)

b=bulboso, c=club, u=coppa, e=uguale, z=rizomorfo, r=radicato, ?=mancante

stalk-surface-below-ring

Superficie gambo sotto l'anello

f=fibroso, y=squamoso, k=setoso, s=liscio

stalk-color-below-ring

Colore gambo sotto l'anello

n=marrone, b=beige, c=cannella, g=grigio, o=arancio, p=rosa, e=rosso, w=bianco, y=giallo

ring-number

Numero di anelli

n=nessuno, o=uno, t=due

spore-print-color

Colore della sporata

k=nero, n=marrone, b=beige, h=cioccolato, r=verde, o=arancio, u=viola, w=bianco, y=giallo

stalk-surface-above-ring

Superficie gambo sopra l'anello

f=fibroso, y=squamoso, k=setoso, s=liscio

stalk-color-above-ring

Colore gambo sopra l'anello

n=marrone, b=beige, c=cannella, g=grigio, o=arancio, p=rosa, e=rosso, w=bianco, y=giallo

veil-color

Colore del velo

n=marrone, o=arancio, w=bianco, y=giallo

ring-type

Tipo di anello

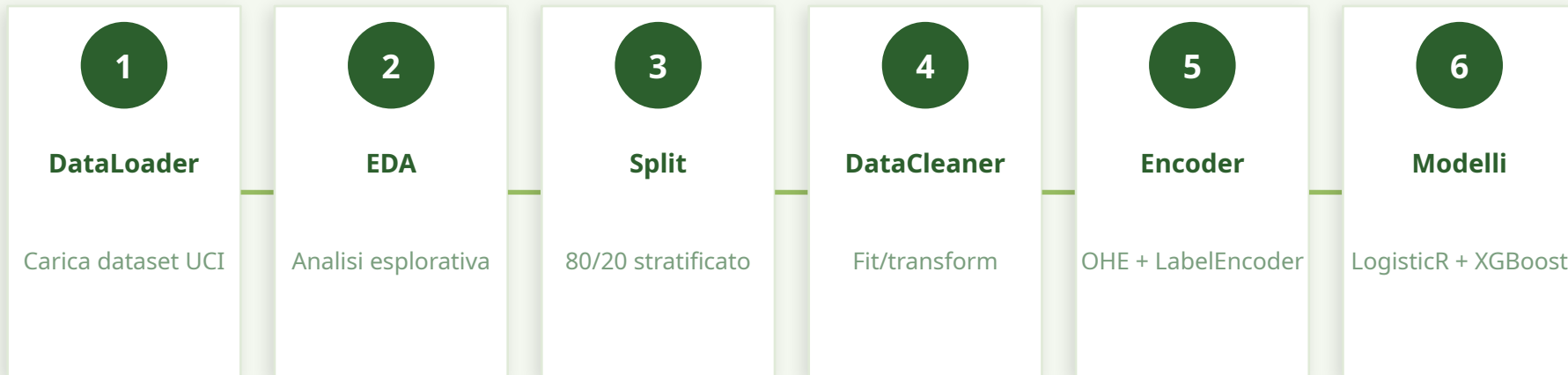
c=ragnatela, e=evanescente, f=svasato, l=grande, n=nessuno, p=pendente, s=guaina, z=zonato

population

Come cresce nella natura

a=abbondante, c=raggruppato, n=numerioso, s=sparso, v=diversi, y=solitario

Pipeline Completa



⚠ Regola anti Data Leakage

DataCleaner ed Encoder vengono sempre fittati SOLO su X_{train} , poi applicati con transform su X_{test} . Questo evita che le statistiche del test influenzino il training.

Split Stratificato 80/20

Cos'è lo stratify?

Senza stratify:

la proporzione edibile/velenoso può variare tra train e test per effetto della casualità.

Con stratify=y:

la proporzione viene garantita identica in entrambi i set.

Dataset completo (8.124)

52% edibile · 48% velenoso

Train 80% (6.499 righe)

52% / 48%

Test 20% (1.625 righe)

52% / 48%

```
train_test_split(  
    X, y, test_size=0.2,  
    random_state=42, stratify=y  
)
```

Data Cleaning — Pattern Fit / Transform

`fit(X_train)`

Impara le statistiche (mediana, moda) SOLO dal train set. Il test non esiste ancora in questo momento.

`transform(X_train)`

Applica le trasformazioni al train usando i valori appresi nella fase di fit.

`transform(X_test)`

Applica le stesse identiche trasformazioni al test. Nessun dato del test ha influenzato le statistiche.

Caso reale nel progetto: stalk-root → 2.480 valori mancanti (30.5%) → sostituiti con 'unknown' (strategia conservativa: preserva l'informazione che il valore mancava)

Encoding delle Variabili

OneHotEncoder → Feature X

Problema:

Le feature sono parole (rosso, blu, verde). Il modello vuole numeri, ma codificare rosso=1, blu=2 creerebbe un ordine falso.

Soluzione:

Ogni categoria diventa una colonna binaria 0/1.

Risultato:

21 colonne → 95 colonne dopo OHE

LabelEncoder → Target y

La target y è 'e' / 'p'.

XGBoost e LogReg si aspettano un numero singolo per classe, NON più colonne.

e → 0 (edibile)

p → 1 (velenoso)

handle_unknown='ignore' nel OneHotEncoder: se nel test appare una categoria non vista nel train, viene ignorata senza far crashare il modello.

Logistic Regression — Teoria

Non è una regressione vera: usa la funzione sigmoide per trasformare un valore lineare in una probabilità tra 0 e 1.

$$P(y=1) = 1 / (1 + e^{-(\beta_0 + \beta_1 x_1 + \dots + \beta_n x_n)})$$

Ideale per: dati lineari, interpretabilità, baseline veloce.

**Output:**

probabilità tra 0 e 1 → soglia 0.5 → classe

**Training:**

minimizza la funzione di perdita

**Interpretabilità:**

i coefficienti β mostrano l'importanza di ogni feature

Logistic Regression — Parametri GridSearchCV

C	0.001 → 100	Inverso della regolarizzazione. C basso = modello semplice (underfitting). C alto = modello complesso (rischio overfitting).
penalty	l1, l2, elasticnet	Tipo di regolarizzazione. L1 azzerava feature irrilevanti (sparse). L2 riduce tutti i pesi. ElasticNet combina entrambe.
solver	liblinear, lbfgs, saga	Algoritmo di ottimizzazione. liblinear per L1, lbfgs per L2 su grandi dataset, saga per elasticnet.
l1_ratio	0.2, 0.5, 0.8	Solo con elasticnet. Bilancia la quota L1 e L2: 0=puro L2, 1=puro L1.
max_iter	5000	Numero massimo di iterazioni prima di fermarsi. Alto per garantire la convergenza su dataset con molte feature.

Logistic Regression — Coefficienti più Significativi

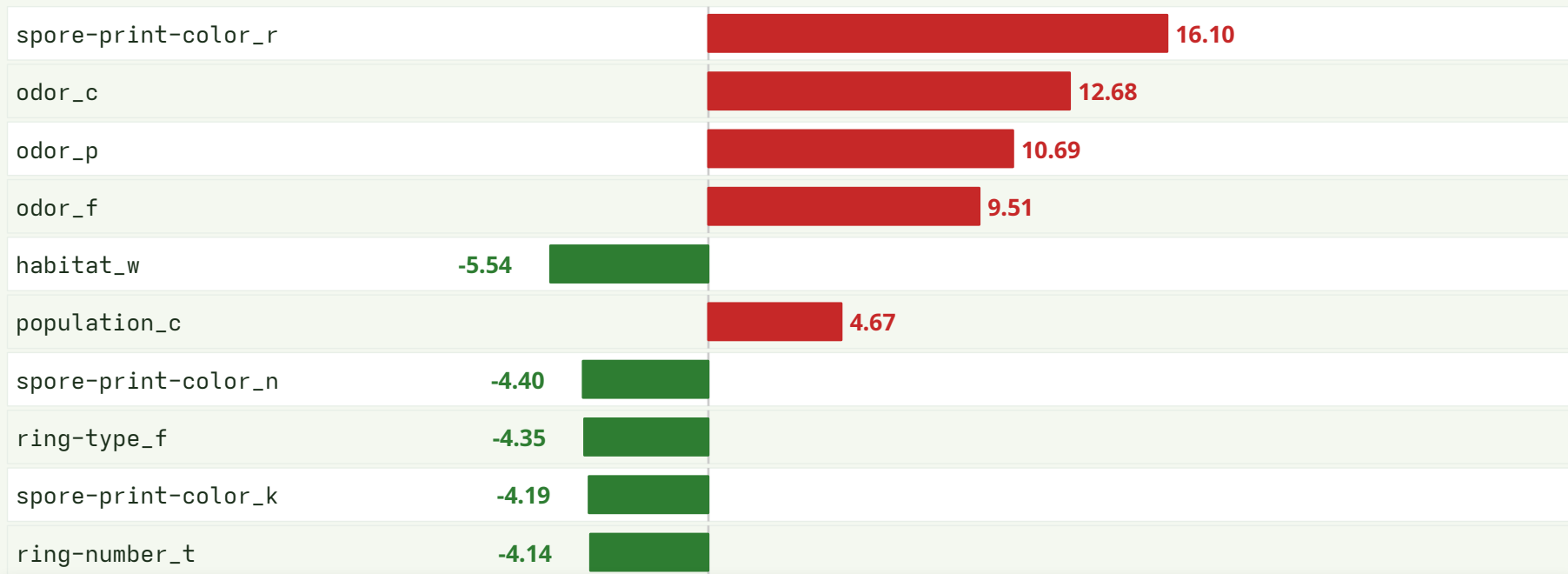


→ velenoso



→ commestibile

Il coefficiente indica quanto ogni feature spinge la predizione verso velenoso (+) o commestibile (-)



odds_ratio = e^{coeff} — spore-print-color_r con coeff 16.10 moltiplica le probabilità di velenoso per ~9.8 milioni se presente.

XGBoost — Teoria

Cos'è XGBoost?

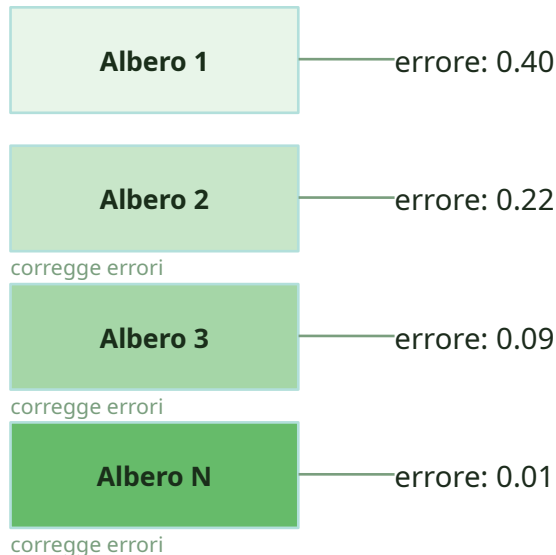
eXtreme Gradient Boosting

Ensemble di alberi decisionali costruiti in sequenza. Ogni albero corregge gli errori del precedente — questa tecnica si chiama Boosting.

Vantaggi:

- Gestisce i missing value nativamente
- Resistente all'overfitting
- Tra i migliori su dati tabulari

Come funziona il Boosting



Predizione finale = somma pesata di tutti gli alberi

XGBoost — Come Corregge gli Errori

Il meccanismo tecnico: ogni albero non impara sui dati originali, ma sui residui — cioè quanto era lontana la predizione precedente dalla realtà.

	Fungo A	Fungo B	Fungo C	Fungo D
Realtà (y)	1 (velenos.)	0 (edibile)	1 (velenos.)	0 (edibile)
Albero 1 predice	0.7	0.6	0.8	0.3
Residui	0.3	0.6 ← alto	0.2	0.3
Albero 2 si concentra su →	–	Fungo B!	–	–

 Analogia: come studiare per un esame. Il secondo ripasso si concentra sulle domande sbagliate, il terzo su quelle ancora sbagliate — alla fine sai rispondere a tutto.

XGBoost — Parametri GridSearchCV

<code>n_estimators</code>	<code>[100, 200, 500]</code>	Numero di alberi. Più alberi = più preciso ma più lento. Si bilancia con <code>learning_rate</code> .
<code>max_depth</code>	<code>[3, 5, 7]</code>	Profondità massima di ogni albero. Valori bassi = underfitting. Valori alti = overfitting.
<code>learning_rate</code>	<code>[0.01, 0.1, 0.3]</code>	Peso di ogni nuovo albero. Basso = impara lentamente ma con più precisione (serve <code>n_estimators</code> alto).
<code>subsample</code>	<code>[0.8, 1.0]</code>	Percentuale di campioni usati per ogni albero. Ridurlo introduce varianza e aiuta contro overfitting.
<code>min_child_weight</code>	<code>[1, 5]</code>	Peso minimo dei nodi foglia. Valori alti rendono il modello più conservativo. (Quanto sono specifiche le regole)

GridSearchCV — Ricerca dei Parametri

Prova automaticamente tutte le combinazioni di parametri e seleziona quella con il miglior score su 5-fold cross validation.

XGBoost

$3 \times 3 \times 3 \times 2 \times 2 = 108$ combinazioni

× 5 fold = 540 addestramenti

n_jobs=-1 → usa tutti i core CPU

Logistic Regression

3 penalty × solver × C × l1_ratio

scoring = 'f1_weighted'

→ privilegia il recall sui velenosi

max_iter=5000 per garantire la convergenza

Metriche di Valutazione

Accuracy

$$\text{corretti} / \text{totale}$$

Percentuale di predizioni corrette totali.

Precision

$$TP / (TP + FP)$$

Dei funghi classificati velenosi, quanti lo erano davvero?

Recall

$$TP / (TP + FN)$$

Di tutti i funghi velenosi reali, quanti ha trovato il modello?

★ più importante per i funghi

F1-Score

$$2 \times P \times R / (P + R)$$

Media armonica tra Precision e Recall. Usata nel GridSearch.

Per i funghi il Recall dei velenosi è critico: è molto peggio classificare un fungo velenoso come commestibile (falso negativo) che il contrario.

Logistic Regression vs XGBoost

Caratteristica	Logistic Regression	XGBoost
Tipo	Modello lineare	Ensemble di alberi
Relazioni non lineari	Non gestisce	Gestisce nativamente
Interpretabilità	Alta (coefficienti)	Bassa (black box)
Valori mancanti	Richiede pulizia	Gestisce nativamente
Velocità training	Veloce	Più lento
Overfitting	Resistente	Richiede tuning
Performance tipica	Buona su dati lineari	Migliore su dati complessi

Nel progetto entrambi i modelli vengono addestrati e confrontati sullo stesso test set per valutare quale performa meglio sul dataset mushroom.

Conclusioni

Pipeline completo

Load → EDA → Split → Clean → Encode → Train → Evaluate

Anti data leakage

Fit sempre su X_{train} , transform su train e test separatamente

Due modelli

Logistic Regression (baseline lineare) e XGBoost (ensemble boosting)

Flask API REST

Ogni step è esposto come endpoint chiamabile in sequenza

Generalizzazione e Libreria Python

Generalizzazione Dataset

Oggi il pipeline è pensato per il dataset mushroom. Il prossimo step è renderlo universale — qualsiasi dataset UCI dovrebbe poter girare senza modifiche al codice.

- Rilevamento automatico del tipo di feature (numerica / categoriale)
- Selezione automatica della strategia di cleaning in base ai dati
- Config file YAML per passare dataset_id, target, drop_cols

Libreria Python

Estrarre DataLoader, DataCleaner, Encoder e modelli in una libreria installabile via pip — riutilizzabile in qualsiasi progetto ML senza ricopiare il codice.

```
pip install mushroom-ml  
  
from mushroom_ml import Pipeline
```

Nuovi Modelli e Production

Nuovi Modelli

Random Forest

Ensemble parallelo — confronto diretto con XGBoost

Neural Network

MLP con PyTorch per relazioni molto complesse

AutoML

TPOT o AutoSklearn per selezione automatica del modello

Stacking

Combinare LR + XGBoost + RF in un meta-modello

Production Ready

CI/CD

GitHub Actions per test automatici ad ogni push

Cloud

Deploy su AWS / GCP con container Docker

Monitor

Monitoraggio data drift — il modello degrada nel tempo?

MLflow

Tracking esperimenti, parametri e metriche nel tempo

Roadmap

Fase 1

Generalizzazione

Config YAML · Qualsiasi dataset UCI · Auto-detection features

Fase 2

Libreria pip

Documentazione · Test coverage > 80%

Fase 3

Nuovi Modelli

Random Forest · Neural Network · Stacking ensemble

Fase 4

Production

CI/CD · Cloud deploy · MLflow · Data drift monitoring